

Part B — Benchmarking & Deployment

Speaking script (~17 min, scale ± 2 min by pacing)

Covers: Part B divider → Desktop client → Evaluation crisis (trilemma) → Hidden-dataset → Security of execution → Extensibility → Submission-flow pipeline diagram → Trust & transparency → Hackathon setup → Results.

[Divider slide — "Benchmarking & deployment"] (~1 min)

Thanks, Keshav. Where Part A made federated learning survive real hospital networks, Part B answers the next question: once a model is trained this way, how do we actually know if it's good — and who gets to verify that claim? That's what BODH.AI's evaluation and deployment layer is built to solve.

[Desktop client slide] (~2 min)

We started by looking at *why* hospitals don't participate in FL platforms today, and the honest answer is: the barrier isn't mathematical, it's operational. Hospital IT staff aren't ML engineers, and asking them to set up Python environments, CUDA drivers, and cryptography libraries is a non-starter.

So we built a productized desktop client — an Electron app that bundles Python, PyTorch, and all the crypto dependencies into a single double-click binary. It only makes outbound connections, so it works behind hospital firewalls and NAT with zero custom IT configuration. One-click authentication, live training visualization, and schema mapping to handle each hospital's data heterogeneity. The goal was: if you can double-click an installer, you can join a national FL network.

[Evaluation crisis / trilemma slide] (~2.5 min)

(gesture to the trilemma triangle)

But participation alone doesn't solve trust. Once a model exists, someone has to evaluate it credibly — and every existing option fails at that in the same structural way. We frame it as a trilemma between three things: openness, reliability, and coverage. In practice, you only ever get two.

Academic benchmarks are open and reliable, but small-scale — they don't cover real population diversity. Hospital chains have the patient coverage, but commercial interests block openness, and limited engineering bandwidth undermines reliability. Startups are agile enough to be open or reliable, but they're coverage-limited just like academia.

BODH.AI is designed to get all three at once: cross-entity federated training gives us coverage, a hidden competitive benchmark gives us reliability, and free public access gives us openness.

[Hidden-dataset slide] (~2 min)

The core idea that makes this possible is strict separation of model development from model evaluation. Contributors train and tune on public data. But scoring happens against a curated clinical dataset that lives hidden on our server — it is never exposed through any API, so there's no way to overfit to it or leak it into training.

That gives us tamper-evident, regulatory-grade evidence rather than a self-reported number. And because we keep enriching that hidden set over time, we avoid benchmark decay — the slow process by which a static leaderboard stops meaning anything once models have effectively memorized it.

[Security of execution slide] (~2 min)

Hidden data is only meaningful if the code touching it can't leak it back out — which means we have to run *untrusted, arbitrary* model code safely. Every submission runs inside a short-lived Docker container with no network access at all. The hidden dataset is mounted read-only, CPU, RAM and runtime are capped, and only the prediction file is ever allowed to leave. The container is destroyed immediately after. So even a malicious or buggy submission has no path to exfiltrate the dataset.

[Extensibility slide] (~1.5 min)

We also didn't want every new model architecture to require us to touch the platform's code. So model definition is fully decoupled from the pipeline — researchers describe their model through a guided form: architecture, preprocessing, output mapping — never code. They upload just a weights file, ONNX, PyTorch, H5, whatever — and a built-in evaluation harness loads and runs it. A brand-new architecture needs zero platform changes.

[Submission-flow diagram slide] (~2.5 min)

(walk the diagram left to right)

Putting all of that together, here's the full pipeline. A researcher uploads their model weights through the React frontend. That request hits our FastAPI backend, which validates the zip, stores it in MinIO, and creates a submission ID. That submission is pushed onto a Redis-backed job queue, and our worker system picks it up when a slot is free — this is what lets us absorb bursts of submissions without the API blocking.

The actual execution happens in two stages inside that sandboxed container we just discussed: stage one runs on a small public "fake" dataset as a fast sanity check — did the model even load, does the output shape match — and only if that passes does stage two run against the real hidden golden dataset. Results — the score, execution time, logs — get written to PostgreSQL, and the frontend polls that to show the researcher their score, their logs, and their leaderboard rank.

[Trust & transparency slide] (~1.5 min)

That leaderboard is public by default — good scores and bad ones. Every entry is cryptographically tied to a specific model hash, a specific dataset version, and a timestamp, so a score can always be traced back to exactly what was run, on exactly what data. And if the hidden dataset gets updated, older entries are automatically flagged as outdated rather than silently left to look current.

[Hackathon setup slide] (~2 min)

(gesture to leaderboard screenshot)

We didn't want to validate any of this only in a lab setting, so we ran it as a live, national-scale hackathon — a real proxy for a national hospital network under chaotic, concurrent load. Over 300 independent teams — data scientists, hospital IT staff, academics — competed across three real clinical problems: diabetic retinopathy, bone-age prediction, and cataract detection. Everyone got public training data; scoring ran against our hidden held-out set on the server.

[Results slide] (~2 min)

The platform held up. 150-plus distinct teams, 450-plus unique submissions, all processed reliably. Average evaluation latency — spin-up to teardown — was under 4.8 seconds, and we had 100% server uptime: no crashes, no database locks, even under concurrent national-scale load. Novel architectures — vision transformers, ensembles, things we hadn't specifically tested for — ran through the pipeline with zero errors, which is the real proof point: the extensibility design held under conditions we didn't fully control.

That's the trust layer of BODH.AI — from here I'll hand back for the combined contributions and what's next.

Total: ~17 minutes at a natural pace. Trim the trilemma or pipeline walkthrough first if you're running long; those two have the most room to compress.